Python Assignment

NAME: HARI VIGNESH L
ROLL NO:251801083
AIDS-B

MCQs

1. **What will be the output?**
   print(len("Python"))

   Answer:
   6

2. **Which of the following is true about infinite loops in Python?**
   They always cause syntax error
   They execute only once
   They run endlessly until stopped
   They automatically terminate

   Answer:
   They run endlessly until stopped

3. In Python, what is the primary use of a for loop?

   Answer:
   To iterate over a sequence

4. **What is the output of the following code?**
   x = 5
   y = 10

   ```
   if x > y:
     print('x is greater')
   else:
     print('x is not greater')
   ```

   Answer:

x is not greater

5. What is the function of the pass statement in loops?

Answer:
Acts as placeholder

Code-1

Problem Statement

Emily is hosting a pizza party. She orders a set number of pizzas, each with a fixed number of slices, and invites some friends. The total slices are shared equally among everyone, including Emily.

After getting her equal share, Emily eats 3 extra slices. Given the number of pizzas, slices per pizza, and number of friends, calculate how many slices Emily eats in total.

Example
Input:
4
8
3

Output:
11

Explanation:
Step 1:
Total Slices
total slices = 4 * 8 = 32
(There are 32 slices in total.)
Step 2:
Each Share
each share = 32 // (3 + 1) = 32 // 4 = 8

(The number 3 + 1 represents the total number of people at the party, including Emily. Emily has 3 friends, so the total number of people is 3 + 1 = 4.)

Step 3:

Emily's Share

you eat = 8 + 3 = 11

(Emily will eat 3 extra slices on top of the equal share, so her total is 8 + 3 = 11.)

Input format :

The first line of input consists of an integer, **p**, representing the number of pizzas ordered.

The second line consists of an integer, **s**, representing the number of slices each pizza has.

The third line consists of an integer, **f**, representing the number of friends invited to the party.

Output format :

The output prints an integer representing how many slices Emily will eat.

Sample test cases :

Input 1 :

4

8

3

Output 1 :

11

Input 2 :

5

6

2

Output 2 :

13

Answer:

```
# You are using Python
a=int(input())
b=int(input())
c=int(input())
t=a*b
e=t//(c+1)
y=e+3
print(y)
```

Code-2:

Problem Statement

Tim is developing a program for a historical research project that involves analyzing and cataloging ancient artifacts. Each artifact is labeled with a Roman numeral representing its historical significance.

Your task is to help Tim convert a Roman numeral into its corresponding integer value.

Valid Roman numerals for this task:

- I = 1
- X = 10
- XL = 40
- L = 50
- XC = 90
- C = 100
- D = 500
- M = 1000

Note: Only the Roman numerals listed above are considered valid. Any other character is invalid. Subtractive notation is allowed only as listed (XL, XC).

Input format :

A single string representing the Roman numeral.

Output format :

If the input is valid, print:

Integer representation: <value>

If an invalid character is encountered, print:

Invalid Roman numeral character: <character>

Refer to the sample output for the formatting specifications.

Code constraints :

The length of the string is between 1 and 10 characters.

The string consists of uppercase letters and valid subtractive combinations only.

Sample test cases :

Input 1 :

MCXII

Output 1 :

Integer representation: 1112

Input 2 :

DLVII

Output 2 :

Integer representation: 557

Input 3 :

VII7

Output 3 :

Invalid Roman numeral character: 7

Answer:

```
roman_values = {
   'I': 1, 'V': 5, 'X': 10, 'L': 50,
   'C': 100, 'D': 500, 'M': 1000
}
roman = input().strip()
for ch in roman:
```

```
        if ch not in roman_values:
                print(f"Invalid Roman numeral character: {ch}")
                exit()
integer_value = 0
prev_value = 0
for ch in reversed(roman):
        current_value = roman_values[ch]
        If current_value < prev_value:
                integer_value -= current_value
        else:
                integer_value += current_value
        prev_value = current_value
print(f"Integer representation: {integer_value}")
```

Code-3:

Problem Statement

Shafir is working on a grid-based problem where he needs to find the number of unique paths from the top-left corner to the bottom-right corner of an

M×N grid. In each step, he can either move right or down. Your task is to compute the total number of distinct paths that lead to the destination, given that movement is restricted to only rightward or downward directions.

Input format :

The first line of input contains an integer M, representing the number of rows of the grid.

The second line of input contains an integer N, representing the number of columns of the grid.

Output format :

The output should be a single integer, representing the number of unique paths from the top-left corner to the bottom-right corner of the grid.

Refer to the sample output for formatting specifications.

Sample test cases :

Input 1 :

3

7

Output 1 :

28

Input 2 :

5

5

Output 2 :

70

Answer:

```
import java.util.Scanner;
class UniquePaths {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    int M = scanner.nextInt();
    int N = scanner.nextInt();
    System.out.println(uniquePaths(M, N));
  }
  public static int uniquePaths(int M, int N) {
    int[][] dp = new int[M][N];
```

```
        for (int i = 0; i < M; i++) {
            dp[i][0] = 1;
        }
        for (int j = 0; j < N; j++) {
            dp[0][j] = 1;
        }
        for (int i = 1; i < M; i++) {
            for (int j = 1; j < N; j++) {
                dp[i][j] = dp[i-1][j] + dp[i][j-1];
            }
        }
        return dp[M-1][N-1];
    }
}
```